Hierarchical Aggregation and Cooperative Caching for Decentralized Federated Learning in UAV-Assisted Internet of Vehicles

Suidan Yuan, Chao Yang, Jiajie Zhou, and Shengli Xie, *Fellow, IEEE*

***Abstract*—Decentralized federated learning (DFL) and efficient content delivery are critical in uncrewed aerial vehicle (UAV)-assisted Internet of Vehicles (IoV). However, the performance is hampered by dynamic topology, energy constraints, and fluctuating computational loads. In this article, we present a unified optimization framework that reuses the DFL aggregation structure for cooperative edge caching in UAV-assisted IoV. In the first stage, we formalize the hierarchical DFL aggregation route as an NP-hard combinatorial problem and solve it with a graph neural network (GNN)-enhanced proximal policy optimization (PPO) algorithm for aggregation route (GPPO-R), which rapidly converges to near-optimal routing structures and achieves the tradeoff balance between synchronization latency and energy consumption. In the second stage, we design a DFL-based multiagent PPO caching strategy (DFL-MAPPO) that leverages the DFL aggregation structure and neighbor-state sharing. It consists of two phases: a long short-term memory (LSTM)-driven proactive caching phase, where the LSTM model is trained via DFL. A real-time multiagent reinforcement learning (MARL) phase for dynamic cache adjustment. Extensive simulations across varying number of UAVs, cache capacities, and user loads demonstrate that GPPO-R and DFL-MAPPO jointly reduce request latency and energy overhead compared to independent variants and classical schemes. The results validate the effectiveness and scalability of the proposed framework in dynamic UAV networks, providing a robust solution for integrated model aggregation and content caching in IoV.**

***Index Terms*—Caching strategy, decentralized federated learning (DFL), Internet of Vehicles (IoV), uncrewed aerial vehicle (UAV).**

NOMENCLATURE

Abbreviation	Expansion
U, V	Sets of all UAVs and vehicles.
Θ	Cache capacity of UAV.
$r_{u_i,v_m}(t)$, $r_{u_i,u_j}(t)$	Transmission rate between UAV u_i and vehicle v_m, and UAV u_i and UAV u_j.
U_B, U_F	Set of backbone nodes and branch nodes.
D_{sync}	Total synchronization latency.
E_{comm}, E_{load}	Total communication and load energy consumptions.
$\mathcal{R}_t$	Set of requests arriving in slot t.
$\mathcal{F}$	Set of all contents.
S_f, S_{req}	Sizes of content f and request information of content f.
$T_{\text{local}}(f,t)$, $T_{\text{neigh}}(f,t)$, $T_{\text{miss}}(f,t)$	Content request latency under local cache hit, neighbor cache hit, and cache miss scenarios.
$T_{\text{total}}(f,t)$	Total content request latency.
$\gamma, \lambda, \epsilon$	Discount factor in PPO, GAE parameter, and PPO clipping threshold in GPPO-R.
ω_g	Parameters of GNN module.
π_θ, π_ϕ	Actor and critic network parameters in GPPO-R.
$\gamma_{\text{MA}}, \lambda_{\text{MA}}, \epsilon_{\text{MA}}$	Discount factor, GAE parameter, and PPO clipping threshold in DFL-MAPPO.
κ	Parameters of LSTM module.
$\pi_{\theta_{\text{MA}}}, \pi_{\phi_{\text{MA}}}$	Actor and critic network parameters in DFL-MAPPO.

I. INTRODUCTION

WITH the advancement of B5G/6G communication technologies, uncrewed aerial vehicles (UAVs) are evolving from flying platforms into edge computing (EC) nodes to support various emerging Internet of Things (IoT) applications [1]. Integrating air-to-ground (A2G) and air-to-air (A2A) links, the self-organizing UAV-assisted Internet of Vehicles (IoV) networks hold great promise for intelligent transportation systems (ITSs), especially for the traffic congestion scenarios. UAVs can provide low-latency network access services for the gathered users efficiently [2], [3]. However, UAV-assisted IoV is confronted with two major challenges. On the one hand, as edge nodes in intelligent distributed systems, UAV networks require efficient learning capabilities to handle large-scale data. On the other hand, the data privacy during such processing should be maintained. federated learning (FL) is elevated as a preferred solution. However, centralized FL introduces a single node of failure and communication bottlenecks, rendering it ill-suited for

Received 10 September 2025; revised 21 October 2025; accepted 28 October 2025. Date of publication 5 November 2025; date of current version 23 December 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62320106008 and Grant 62003094 and in part by Guangdong Basic and Applied Basic Research Foundation under Grant 2024A1515012745. *(Corresponding author: Chao Yang.)*

The authors are with the School of Automation and Guangdong Provincial Key Laboratory of Intelligent Systems and Optimization Integration, Guangdong University of Technology, Guangzhou 510006, China (e-mail: chyang513@gdut.edu.cn).

Digital Object Identifier 10.1109/JIOT.2025.3629070

2327-4662 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:23:03 UTC from IEEE Xplore. Restrictions apply.

highly dynamic UAV networks. Decentralized federated learning (DFL) is proposed for eliminating the central server by enabling direct model aggregation among UAVs [4], [5].

Wireless edge caching increases the users' satisfaction with minimal task processing latency and energy consumption. UAVs can function as a content delivery network (CDN) by proactively caching user-requested contents, such as high-definition maps, real-time video streams, and software update packages, thereby enhancing the overall system quality of experience (QoE) [6]. However, the high mobility of UAVs makes traditional static caching strategies ineffective, such as least recently used (LRU) and least frequently used (LFU) [7]. Researchers used distributed multiagent reinforcement learning (MARL) to optimize edge caching decisions [8], [9]. The cache hit rates were improved. Moreover, some recent works modeled UAV trajectories and caching decisions jointly, leading to collaborative optimization and significant performance improvements [10]. Despite the significant benefits of UAV-assisted IoV, DFL and edge caching face persistent challenges. UAVs suffer from continuous energy depletion during flight, hovering, and computation, and from fluctuating computational loads and intermittent communication links. The harsh wireless channel conditions and link instability compromise reliable model aggregation and invalidate conventional caching strategies [11], [12]. The hierarchical backbone–branch topology, adopted in vehicular ad hoc networks (VANETs) [13] and wireless sensor networks (WSNs) [14], offers a promising direction by organizing nodes into clusters with virtual backbones of cluster heads, thereby balancing load and improving end-to-end performance [15]. While existing FL research focuses on model compression, gradient quantization, and hierarchical FL (HFL) [16], [17], [18], a notable gap persists in utilizing communication infrastructure reusability, particularly how DFL architectures could be leveraged for additional network functions such as content caching. Part of existing approaches relies on centralized coordination, and caching is often performed in isolation, neglecting opportunities for cooperative content sharing among neighboring devices.

To address the limitations, we propose a unified framework that integrates DFL with cooperative edge caching. The hierarchical DFL aggregation architecture, originally developed for model synchronization, is used for efficient content dissemination. We first design a DFL hierarchical aggregation structure and propose a novel routing algorithm among UAVs to balance model synchronization latency, communication energy consumption, and node load. We also propose a graph neural network (GNN)-enhanced proximal policy optimization (PPO) algorithm, which adapts to changing network topology to construct near-optimal hierarchies. Then, leveraging the aggregation topology, we deploy a two-phase caching scheme: a long short-term memory (LSTM)-driven proactive caching phase that predicts content popularity and preloads caches via inter-UAV exchanges, followed by a multiagent PPO (MAPPO)-based cache adjacent phase that dynamically refines cache placement in real time to meet users' requests.

The main innovative contributions are as follows.

1) We propose a unified framework that uses the dynamic DFL aggregation structure for cooperative edge caching.
2) We develop a novel routing algorithm that integrates GNN with PPO to dynamically learn and construct a hierarchical aggregation structure for DFL, achieving the tradeoff between synchronization latency and energy consumption.
3) We present an MAPPO-based caching strategy that leverages the DFL aggregation structure and neighbor-state sharing in a two-phase process.

In addition, simulations over varying numbers of UAVs, cache sizes, and user loads show that the proposed joint routing and caching scheme outperforms standalone and classical methods, demonstrating its effectiveness and scalability in UAV-assisted IoV. The rest of this article is organized as follows. Section II reviews related work. Section III presents the system model. Section IV details the DFL aggregation structure. Section V outlines the cooperative caching strategy. Section VI presents simulation results evaluating the proposed method. Finally, Section VII concludes this article.

II. Related Work

A. DFL for UAV-Assisted IoV

In UAV-assisted IoV, DFL enhances system robustness and scalability by eliminating reliance on a central server through peer-to-peer collaborative training among nodes.

Regarding the construction of efficient system architectures and aggregation strategies, Tummala et al. [19] proposed a clustered pseudo-hierarchical framework that improves the scalability of large-scale UAV networks through two-tier aggregation. Pan et al. [20] designed a serverless FL architecture for UAV networks called DFL-UN, which introduces two decentralized parameter aggregation methods that effectively manage local training rounds and communication overhead. Furthermore, Tong et al. [21] proposed a blockchain-based hierarchical FL scheme that enhances update integrity and aggregation efficiency in fully decentralized UAV-IoT environments. To adapt to highly dynamic network environments and address node heterogeneity, Reza et al. [22] and Chen et al. [23] leveraged mobility awareness to mitigate data heterogeneity and improved training efficiency. Tong et al. [24] proposed AirFRL, a topology-aware DFRL framework specifically designed for UAV networks. Additionally, the ADFL framework proposed by Wang et al. [25] used reinforcement learning (RL) to generate adaptive collaboration strategies for heterogeneous nodes, addressing dynamic challenges.

In terms of improving communication, computation, and energy efficiency, Tan et al. [26] introduced the SC1BCS-WFL framework, which reduces communication overhead. Solat et al. [27] designed the E-SFL framework, focusing on providing an energy-efficient mobile traffic prediction method to alleviate the training burden. Furthermore, to incentivize node participation and ensure update quality, Hu et al. [28] applied a two-layer Stackelberg game theory model to design incentive mechanisms and incorporated a reputation system to ensure the quality of model updates.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:23:03 UTC from IEEE Xplore. Restrictions apply.

B. Content Caching for UAV-Assisted IoV

In UAV-assisted IoV, constructing a hierarchical content caching network among UAVs, roadside units (RSUs), and vehicles can alleviate bandwidth pressure and reduce service latency. Yu et al. [29] proposed a trajectory-prediction-based caching scheme that exploits spatiotemporal vehicle mobility for content placement. Due to limited UAV storage, cooperative caching across UAVs, RSUs, and vehicles is adopted to construct large-scale distributed caches, thereby improving hit rates and system resilience. Yang et al. [30] coordinated this via a coalition-game-based distributed clustering algorithm. In addition, Yan et al. [31] investigated UAV-assisted content caching for human-centric IoV to improve user-perceived relevance, while Zhou et al. [32] focused on jointly optimizing service caching and task offloading based on user pReferences in UAV-assisted mobile EC to reduce latency.

Deep reinforcement learning (DRL) has been widely applied to jointly optimize cache placement and UAV trajectory in UAV-assisted IoV. Wang et al. [33] proposed a CTDR framework combining differential evolution for caching and MAPPO for trajectories to cut energy use. Diallo et al. [34] introduced OHDRL to co-optimize UAV trajectory, content acquisition, power allocation, and placement. Long et al. [35] applied an improved artificial fish swarm algorithm to lower total acquisition energy by balancing popularity and node energy. For efficient management, Ou et al. [36] designed a DRL-based cooperative caching method, while Liu et al. [37] proposed a joint caching/replacement strategy that adapts to content popularity and vehicle location.

While prior studies have advanced DFL and content caching, most DFL efforts focus on improving aggregation efficiency and scalability, without utilizing learned topologies for network-level coordination. Conversely, caching methods emphasize cooperative placement and trajectory design, often treating learning and caching as isolated processes. To address these limitations, we innovatively repurpose the DFL-optimized aggregation structure to drive a cooperative caching mechanism. Notably, our aggregation structure is specifically designed to balance the energy consumption and synchronization delay. The cooperative caching mechanism combines LSTM-based prediction with MARL-based real-time adjustment.

III. System Model

A. System Structure

The UAV-assisted two-tier IoV system architecture is shown in Fig. 1. It includes UAVs, $U = \{u_1, u_2, \ldots, u_M\}$, each with an EC server and connected to the cloud server (CS), and vehicles $V = \{v_1, v_2, \ldots, v_N\}$. The top tier consists of UAVs deployed above roads, while the bottom tier comprises vehicles within the UAV communication coverage. The set of vehicles served by UAV u_i is $V_{u_i} = \{v_{u_i,1}, \ldots, v_{u_i,j}, \ldots, v_{N_{u_i}}\}, u_i \in U$. Each vehicle is served by one UAV, and UAVs are deployed according to the spatial distribution of vehicles to ensure complete coverage under communication range constraints. When road congestion occurs and local computational demands from vehicle users surge, the ITS center dispatches UAVs to serve vehicle users in the affected area. The CS and UAVs, and the UAVs and vehicles are connected via wireless links. Vehicle users have local computing capacity, and the CS can handle all content requests. The caching capacity of each UAV is denoted as Θ. The main notations are presented in Nomenclature.

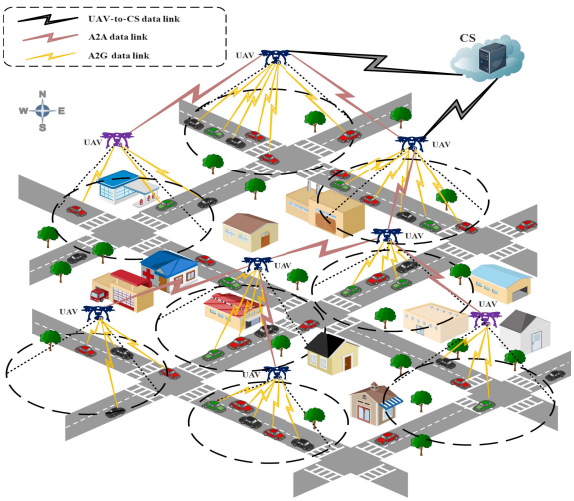


Fig. 1. System architecture.

B. Communication Model

The system supports two communication modes: A2G links between UAVs and ground vehicles, and A2A links among UAVs [38]. In urban environments, both A2G and A2A channels experience line-of-sight (LoS) and non-LoS (NLoS) conditions. We model the LoS and NLoS probabilities for each link type and assume identical uplink and downlink transmission rates across all links. Denote the position of UAV u_i as (x_{u_i}, y_{u_i}, h) and that of vehicle v_j as $(x_{v_j}, y_{v_j}, 0)$. The distance between UAVs u_i and u_j is $d_{u_i,j}$.

1) A2G Communication Model: Due to obstructions caused by urban buildings, A2G links can be either LoS or NLoS connections. The LoS probability, $P_{\text{LoS}}^{u_i,v_m}$, depends on the elevation angle φ_{u_i,v_m} between the UAV and the vehicle and is given by

$$P_{\text{LoS}}^{u_i,v_m}(t) = \frac{1}{1 + a_1 \exp\left(-a_2\left(\varphi_{u_i,v_m}(t) - a_1\right)\right)} \tag{1}$$

where a_1 and a_2 are urban environment parameters. The elevation angle $\varphi_{u_i,v_m} = 180/\pi \arcsin(h_{u_i}/d_{u_i,v_m})$, where h_{u_i} is the UAV height and $d_{u_i,v_m}(t)$ is the instantaneous distance between the UAV and the vehicle.

The total path loss for the A2G link L_{u_i,v_m} is a probability-weighted average of the LoS and NLoS losses as follows:

$$L_{u_i,v_m}(t) = P_{\text{LoS}}^{u_i,v_m}(t)\, L_{\text{LoS}}^{u_i,v_m}(t) + \left(1 - P_{\text{LoS}}^{u_i,v_m}(t)\right) L_{\text{NLoS}}^{u_i,v_m}(t) \tag{2}$$

$$L_{\text{LoS}}^{u_i,v_m}(t) = 20\log_{10}\left(\frac{4\pi\vartheta d_{u_i,v_m}(t)}{c}\right) + \eta_{\text{LoS}} \tag{3}$$

$$L_{\text{NLoS}}^{u_i,v_m}(t) = 20\log_{10}\left(\frac{4\pi\vartheta d_{u_i,v_m}(t)}{c}\right) + \eta_{\text{NLoS}} \tag{4}$$

where ϑ is the carrier frequency, c is the speed of light, and η_{LoS} and η_{NLoS} are additional loss factors. Then, the

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:23:03 UTC from IEEE Xplore. Restrictions apply.

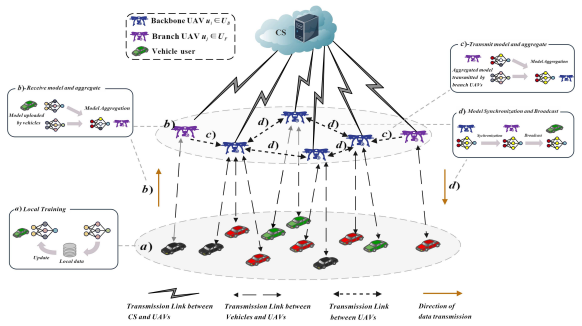


Fig. 2. DFL training model.

instantaneous data transmission rate $r_{u_i,v_m}(t)$ can be expressed as

$$r_{u_i,v_m}(t) = B\log_2\left(1+\mathrm{SNR}_{u_i,v_m}(t)\right) \tag{5}$$

where $\mathrm{SNR}_{u_i,v_m}(t) = P_{tx,v_m}G_{u_i,v_m}(t)/\sigma^2$. P_{tx,v_m} is the vehicle's transmit power, $G_{u_i,v_m} = 10^{-L_{u_i,v_m}(t)/10}$ is the channel gain, B is bandwidth, and σ^2 is the noise power.

2) A2A Communication Model: A2A links are assumed to operate predominantly under LoS conditions. The path loss between UAVs u_i and u_j is modeled as

$$L_{u_i,u_j}(t) = 20\log_{10}\left(\frac{4\pi\vartheta d_{u_{i,j}}(t)}{c}\right) \tag{6}$$

where $d_{u_{i,j}}(t)$ represents the Euclidean distance between the UAVs and γ is the environmental constant. The transmission rate between UAVs u_i and u_j is

$$r_{u_i,u_j}(t) = B\log_2\left(1+\mathrm{SNR}_{u_i,u_j}(t)\right) \tag{7}$$

where $\mathrm{SNR}_{u_i,u_j}(t) = P_{tx,u_i}G_{u_i,u_j}(t)/\sigma^2$, P_{tx,u_i} represents the transmission power of UAV u_i, and $G_{u_i,u_j}(t) = 10^{-L_{u_i,u_j}(t)/10}$ denotes the channel gain, which is determined from the path loss $L_{u_i,u_j}(t)$. Set that UAVs are assigned nonoverlapping frequency resources by a central scheduler prior to mission execution, thereby minimizing simultaneous co-channel transmissions [39], [40], [41].

C. DFL Aggregation and Synchronization Overhead Model

This section details the entire process of performing a DFL global model update within the given system architecture. Based on this process, an overhead calculation model is established. UAVs in the network are classified into backbone nodes U_B and branch nodes U_F, $U_B \cup U_F = U$.

1) DFL Training Model: In DFL, a two-tier aggregation mechanism is used to collect model updates from users. The DFL training model is illustrated in Fig. 2, with its depicted steps directly corresponding to the following four phases of a single DFL training round.

a) Local training: All participating vehicle users independently train on their local datasets to obtain the respective model updates.

b) First-tier aggregation: Each vehicle uploads its local model to the nearest UAV, which acts as a first-tier aggregator by combining all received vehicle models into a preliminary local aggregate.

c) Second-tier aggregation: After completing the first-tier aggregation, all branch nodes U_F transmit their local aggregated models to the corresponding backbone nodes. The backbone nodes, acting as second-tier aggregators, combine the local models from all their branch nodes and their own.

d) Global model synchronization and broadcast: Once the global model is updated by any backbone node, it is disseminated across the entire network. Subsequently, each UAV node distributes the updated model to the ground vehicles within its coverage area, initiating the next round of DFL.

2) Communication Overhead Model: Based on the aforementioned process, the communication overhead of complete model synchronization and broadcast can be broken down into four main phases.

a) Vehicle user upload model to UAV: All ground vehicles upload their local models concurrently to the nearest UAV. The synchronization latency D_{sync_1} in this phase depends on the last vehicle to complete the upload. The total communication energy consumed by all vehicles during the upload phase is denoted as E_{comm_1}.

b) Branch to backbone aggregation: All branch nodes concurrently upload their local aggregated models to their respective backbone nodes. The synchronization latency D_{sync_2} depends on the last branch node to finish model transmission, representing the maximum A2A transmission latency in this phase. The communication energy consumption E_{comm_2} is the total A2A communication energy consumed by all branch nodes during the upload.

c) Backbone internal model synchronization: The aggregated model is broadcast and synchronized among all backbone nodes. The latency D_{sync_3} and energy consumption E_{comm_3} are calculated similar to the previous phases.

d) Global model system broadcast: The last backbone node to finish synchronization broadcasts the global model to the entire system. The latency D_{sync_4} and energy consumption E_{comm_4} are calculated in the same way as in the previous phases as

$$D_{\mathrm{sync}} = D_{\mathrm{sync}_1} + D_{\mathrm{sync}_2} + D_{\mathrm{sync}_3} + D_{\mathrm{sync}_4} \tag{8}$$

$$E_{\mathrm{comm}} = E_{\mathrm{comm}_1} + E_{\mathrm{comm}_2} + E_{\mathrm{comm}_3} + E_{\mathrm{comm}_4}. \tag{9}$$

3) Load Overhead Model: Since both backbone and branch nodes perform aggregation tasks, their load overheads are modeled separately.

a) Load energy consumption of backbone nodes $E_{load}(u_i)$, $u_i \in U_B$: It arises from both the vehicles it serves and the branch nodes it manages. Since backbone nodes must communicate with their associated vehicles and coordinate with branch nodes, their load energy consumption increases due to the combined costs of vehicular transmissions and internode coordination as follows:

$$E_{\mathrm{load}}(u_i) = \left(E_0 + \alpha\cdot N_{u_i} + \beta\cdot M_{u_i}\right)\cdot\zeta_i \tag{10}$$

where E_0 is the base energy consumption of a node; α and β are the coefficients of vehicle load and branch load, respectively; and ζ_i is the heterogeneity factor for backbone nodes. N_{u_i} and M_{u_i} denote the number of vehicles and branch

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:23:03 UTC from IEEE Xplore. Restrictions apply.

nodes served by u_i. A larger N_{u_i} indicates that UAV u_i already serves many vehicles, and if it is selected as a backbone node, it must additionally communicate with multiple branch nodes. This results in higher overall energy consumption. Therefore, UAVs with high N_{u_i} values are less likely to be selected as backbones from an energy efficiency perspective. Nevertheless, the final backbone selection is determined by a combination of factors, including overall network connectivity, which are further discussed in Section IV.

b) Load energy consumption of branch nodes $E_{load}(u_j)$, $u_j \in U_F$: It is solely determined by the number of served vehicles, denoted as N_{u_j}

$$E_{\text{load}}\left(u_j\right) = \left(E_0 + \alpha \cdot N_{u_j}\right) \cdot \zeta_j \tag{11}$$

where ζ_j denotes the node-specific heterogeneity factor for branch nodes.

D. Content Request Latency Model

In the system model, we assume that each vehicle's request follows the *Zipf* distribution [42]. When user v_g requests content $f \in \mathcal{R}_t$, the system first searches for the nearest UAV. Depending on the cache location of file f, its end-to-end access latency can be categorized into three main scenarios. In all cases, we neglect the request uploading time but include the time for UAVs to exchange request messages. Define the size of file f as S_f and the size of the request information as S_{req}.

1) Local Cache Hit: The requested file f is already cached in the local UAV serving the user. In this case, content acquisition requires no additional network cooperation; the serving UAV directly delivers it to the user via A2G link. The access latency $T_{\text{local}}(f,t)$ is solely the file transmission time as

$$T_{\text{local}}\left(f,t\right) = \frac{S_f}{r_{u_i,v_m}\left(t\right)}. \tag{12}$$

2) Neighbor Cache Hit: If the local UAV u_l does not cache file f, the system queries neighboring UAVs within its communication range. If any of these neighboring UAVs stores the file, the access latency $T_{\text{neigh}}(f,t)$ can be expressed as

$$T_{\text{neigh}}\left(f,t\right) = \frac{S_{\text{req}}}{r_{u_l,u_n}\left(t\right)} + \frac{S_f}{r_{u_l,u_n}\left(t\right)} + \frac{S_f}{r_{u_l,v_g}\left(t\right)} \tag{13}$$

where $S_{\text{req}}/r_{u_l,u_n}(t)$ is the time for the local UAV u_l to forward the request to a neighboring UAV u_n that holds the content via an A2A link. $S_f/r_{u_l,u_n}(t)$ represents the time for the neighboring UAV u_n to transmit the requested content back to the local UAV u_l via an A2A link. $S_f/r_{u_l,v_g}(t)$ is the time consumption for the local UAV u_l to send the content to user v_g via A2G link.

3) Cache Miss: When no UAV in the local cluster caches file f, the access latency $T_{\text{miss}}(f,t)$ is

$$T_{\text{miss}}\left(f,t\right) = \frac{S_{\text{req}}}{r_{u_l,u_n}\left(t\right)} + T_{\text{cloud}} + \frac{S_f}{r_{u_l,v_g}\left(t\right)}. \tag{14}$$

In this case, the request must be retrieved from the CS via the local UAV. The CS retrieves the file and sends it back to the local UAV and then delivers it to the user via an A2G link. $S_{\text{req}}/r_{u_l,u_n}(t)$ represents the request upload latency. T_{cloud} represents the cloud service latency. $S_f/r_{u_l,v_g}(t)$ is the content download latency, which is the time consumption for the local UAV u_l to deliver the content to the ground user v_g via an A2G link.

In summary, the total latency is calculated as

$$T_{\text{total}}\left(f,t\right) = \begin{cases} T_{\text{local}}\left(f,t\right), & \textbf{if} \quad \text{local cache hit} \\ T_{\text{neigh}}\left(f,t\right), & \textbf{if} \quad \text{neighbor cache hit} \\ T_{\text{miss}}\left(f,t\right), & \textbf{if} \quad \text{cache miss.} \end{cases} \tag{15}$$

IV. DFL Aggregation Structure

A. Problem Formulation

The network topology formed by UAVs is denoted as an undirected graph $G = (U, E)$. Here, U is the set of all UAVs and E is the set of feasible communication links between nodes. The network connectivity is represented by an adjacency matrix $C \in \{0,1\}^{M\times M}$, where $C_{ij} = 1$ indicates the existence of a direct communication link between nodes u_i and u_j, $(u_i, u_j \in U)$ at the current time, and 0 otherwise.

Building upon this graph model, we aim to address a core challenge in DFL aggregation structure design: *how to find a topology in dynamic networks that optimally balances model synchronization latency, communication energy consumption, and node load?* To solve this, we formalize the task of constructing the optimal aggregation structure as a combinatorial optimization problem **P1**

$$\begin{aligned} \textbf{P1}: \quad & \min \left\{ w_d D_{\text{sync}}\left(U_P\right) + w_e E_{\text{comm}}\left(U_P\right) + w_l E_{\text{load}}\left(U_P\right) \right\} \\ & \text{s.t. } C1: k_{\min} \le |U_B| \le k_{\max} \\ & \quad C2: \kappa\left(L\left(G\left[U_P\right]\right)\right) > 0 \\ & \quad C3: \forall u_i \in U_B, E_{\text{rem}}\left(u_i\right) \ge E_{\text{threshold}} \\ & \quad C4: \forall u_j \in U_F, \sum_{u_i \in U_B} C_{i,j} \ge 1 \\ & \quad C5: U_B \cap U_F = \emptyset, U_B \cup U_F = U \end{aligned}$$

where $D_{\text{sync}}(U_P)$, $E_{\text{comm}}(U_P)$, and $E_{\text{load}}(U_P)$ represent the total system latency, communication energy consumption, and load energy consumption under the given aggregation structure, respectively. Their detailed calculations are provided in Section III-C. The weights w_d, w_e, and w_l preset and satisfy $w_d + w_e + w_l = 1$. For problem **P1**, C1 constrains the number of backbone nodes to be between $k_{\min}$ and $k_{\max}$. C2 ensures that the selected backbone nodes form a connected subgraph, which is a topological requirement for effective global model synchronization and propagation, formalized through algebraic connectivity κ. C3 stipulates that only UAVs with remaining energy E_{rem} above a threshold E_g qualify as backbone nodes, thus ensuring network stability and endurance. C4 guarantees that every branch node is connected to at least one backbone node, ensuring full participation in the DFL aggregation and preventing isolated branch nodes. C5 ensures that the set of backbone nodes U_B and the set of branch nodes U_F are disjoint, and that their union covers the entire network.

B. GNN-Enhanced PPO Solution

P1 is a complex combinatorial optimization problem, which is NP-hard. Traditional methods are inadequate for handling

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:23:03 UTC from IEEE Xplore. Restrictions apply.

the highly dynamic environment of UAV networks, where both topology and node states change frequently. Therefore, we model the problem as a Markov decision process (MDP), with the core elements, a five-tuple $\{S, A, P, R, \gamma\}$, defined as follows.

1) State Space: To enable the RL agent to comprehend the system's topological structure, we adopt a GNN as the core state representation module. Motivated by the inherently graph-structured system state, the GNN performs localized message passing and neighborhood aggregation to produce permutation-invariant relational embeddings. The GNN excels at processing graph-structured data and is capable of encoding discrete adjacency relationships, thereby improving sample efficiency and training stability [43], [44], [45], [46]. At decision time t, the environment state S_t is a tuple comprising the GNN-encoded output and other state information

$$S_t = \{\mathbf{E_t}, \mathbf{I_t}, \mathbf{C_t}, g_t, b_t, f_t, l_t\} \tag{16}$$

where $\mathbf{E_t}$ is the node embedding matrix produced by the GNN to compress each UAV's local topological information into a d-dimensional feature. $\mathbf{l_t}$ is the routing matrix encoding the logical aggregation links in the structure. $\mathbf{C_t}$ is the physical adjacency matrix indicating instantaneous connectivity among UAVs. g_t is the global graph embedding vector generated by the GNN to summarize the entire network's topology. b_t is the backbone indicator vector marking which UAVs serve as backbone nodes. f_t is the branch indicator vector marking which UAVs serve as branch nodes. l_t quantifies each UAV's current load, which incorporates factors such as the number of connected vehicles. It enables the RL agent to consider load balance during backbone selection. These ordered components together form a compact, information-rich representation that enables the RL agent to make informed aggregation decisions.

2) Action Space: At each decision step t, the agent's action a_t is drawn from the discrete action set $A = U \cup \{a_{\text{stop}}\}$, where choosing a UAV $u_i \in U$ adds that node to the backbone network and selecting a_{stop} terminates the construction. The sequential building process should satisfy feasibility constraints: the newly selected node must be adjacent to the existing backbone subgraph, no node may be added more than once, and the final backbone size must lie between the prescribed $\{k_{\min}, k_{\max}\}$. Any action violating these rules is deemed invalid and is penalized.

3) Reward: When the agent chooses the a_{stop} action or reaches the maximum number of steps, the episode ends, and a final reward r_t is obtained, aiming to minimize the overall overhead, as follows:

$$r_t = -w_d D_{\text{sync}} - w_e E_{\text{comm}} - w_l E_{\text{load}} - p_{\text{invalid}} \tag{17}$$

where p_{invalid} is a fixed penalty applied whenever the agent attempts an invalid action.

4) Algorithm Design: To solve this MDP, we employ the PPO algorithm and design a neural network architecture based on the actor-critic framework. In this architecture, a GNN acts as a shared front-end feature extractor, responsible for extracting key embedding features from raw states. These features are then fed into two distinct networks: a gated recurrent unit (GRU) serving as the Actor network, which incorporates historical action sequences to output the probability distribution of the current action, $\pi_\theta(a_t \mid s_t)$; and a multilayer perceptron (MLP) serving as the critic network, which outputs an estimate of the current state's value $V_\phi(s_t)$.

The training process centers around PPO's unique update mechanism, which stabilizes learning by constraining the probability ratio between new and old policies. To calculate the advantage function A_t, which quantifies the relative performance of the chosen action compared to the average performance of the current policy, we use generalized advantage estimation (GAE). This function's calculation relies on the temporal difference (TD) error δ_t

$$\delta_t = r_t + \gamma V_\phi\left(s_{t+1}\right) - V_\phi\left(s_t\right). \tag{18}$$

The GAE advantage function A_t is a weighted, discounted sum of all future TD errors

$$A_t = \sum_{x=0}^{T-t-1} (\gamma\lambda)^x \delta_{t+x} \tag{19}$$

where $\lambda \in [0, 1]$ is the smoothing parameter for the GAE algorithm.

Subsequently, the policy network and value network are updated by optimizing their respective loss functions. The policy network aims to maximize a clipped surrogate objective function $L^{\text{CLIP}}(\theta)$, which ensures training stability by constraining the probability ratio, $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$

$$\begin{aligned} &L^{\text{CLIP}}(\theta) \\ &= \mathbb{E}_t\left[\min\left(\rho_t(\theta) A_t, \text{clip}\left(\rho_t(\theta), 1-\epsilon, 1+\epsilon\right) A_t\right)\right]. \end{aligned} \tag{20}$$

The value network is updated by minimizing the mean squared error $L^{\text{VF}}(\phi)$ between its predicted value and the actual return estimate

$$L^{\text{VF}}(\phi) = \mathbb{E}_t\left[\left(V_\phi(s_t) - \left(A_t + V_{\phi_{\text{old}}}(s_t)\right)\right)^2\right]. \tag{21}$$

The complete training process follows the standard RL paradigm, involving collecting experience trajectories, computing advantage functions, and performing multiple rounds of optimization on the networks. The structural framework of the proposed GNN-enhanced PPO algorithm for aggregation route (GPPO-R) is illustrated in Fig. 3, and its detailed steps are summarized in Algorithm 1. Nomenclature summarizes the main initialization parameters for the algorithms used in this article.

V. DFL-Based MAPPO Collaborative Caching Strategy

A. Problem Formulation

In this section, we apply the proposed DFL aggregation structure to collaborative content caching tasks, aiming to enhance overall QoE. In decentralized UAV networks, disordered and independent decisions of each UAV can easily lead to duplicate caching of popular content, while other critical contents are missing, resulting in wasted caching resources and degraded user experience. Designing a collaborative caching strategy is crucial.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:23:03 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 GNN-Enhanced PPO Algorithm for Aggregation Route (GPPO-R)

Input: U.

Initialization: γ, λ, ϵ, $N_{\max}$, $T_{\max}$, $M_{\max}$, $k_{\min}$, $k_{\max}$, ζ_i, ζ_j, p_{invalid}, ω_g, π_θ,π_ϕ.

Procedure:

Extract node embeddings matrix $\mathbf{E}_0$ and graph embedding vector g_0 based on GNN;
for $n = 1$ to $N_{\max}$ **do**
  for $t = 1$ to $T_{\max}$ **do**
    Construct state $\{\mathbf{E}_t, \mathbf{I}_t, \mathbf{C}_t, g_t, b_t, f_t, l_t\}$;
    Select action a_t based on actor network;
    Execute action, receive reward r_t and state s_{t+1};
    Store experience $(S_t, a_t, r_t, V(S_t), \log \pi_\theta(a_t \mid S_t))$;
  end for
  Compute TD errors and advantages based on Eq. (18) and Eq. (19) with returns $R_t = \hat{A}_t + V_\phi(s_t)$.
  for $m = 1$ **to** $M_{\max}$ **do**
    Compute probability ratio $\rho_t(\theta)$;
    Compute clipped surrogates: $\xi_{\text{uc},t} = \rho_t(\theta)\,\hat{A}_t$ and
    $\xi_{\text{c},t} = \text{clip}\big(\rho_t(\theta), 1-\epsilon, 1+\epsilon\big)\,\hat{A}_t$.
    Compute actor and critic loss based on Eq. (20) and Eq. (21);
    Update parameters θ, ϕ by descending $\nabla_\theta L^{\text{CLIP}}$ and $\nabla_\phi L^{\text{VF}}$.
  end for
end for

Output: Optimized routing path U_P^*.

At slot t, we characterize system performance using two key metrics: the cache hit rate H_t and the average request latency L_t as

$$H_t = \frac{1}{|\mathcal{R}_t|} \sum_{f \in \mathcal{R}_t} \mathbf{1}_{\text{hit}}(f,t) \tag{22}$$

$$L_t = \frac{1}{|\mathcal{R}_t|} \sum_{f \in \mathcal{R}_t} T_{\text{total}}(f,t) \tag{23}$$

where $|\mathcal{R}_t|$ denotes the total number of requests observed in slot t and

$$\mathbf{1}_{\text{hit}}(f,t) = \begin{cases} 1, & \text{if hit}(f,t) \\ 0, & \text{otherwise.} \end{cases} \tag{24}$$

To find the optimal collaborative caching strategy to maximize users' satisfaction, the optimization problem can be expressed as **P2**

$$\begin{aligned} \mathbf{P2}: \quad & \max_{\{\Upsilon_{j,f}\}} \; w_h H_t - w_{\text{rl}} L_t \\ \text{s.t. } C6: \quad & \sum_{f \in \mathcal{F}} \Upsilon_{j,f} S_f \le \Theta_j \quad \forall j \\ C7: \quad & \Upsilon_{j,f} \in \{0,1\} \quad \forall j,\, f \\ C8: \quad & \sum_{u_j \in U_B} C_{i,j} \ge 1 \quad \forall u_i \in U \end{aligned}$$

where $\mathcal{F}$ is the set of all available contents, S_f denotes the content size, and Θ_j denotes the cache capacity. w_h and w_{rl} are nonnegative weights, $w_h + w_{\text{rl}} = 1$. Constraint C6 enforces the cache-capacity limit, C7 specifies the binary nature of the placement variables, $\Upsilon_{j,f} = 1$ indicates that content is cached in the current slot, and C8 ensures that each UAV u_i has at least one connection to a backbone node.

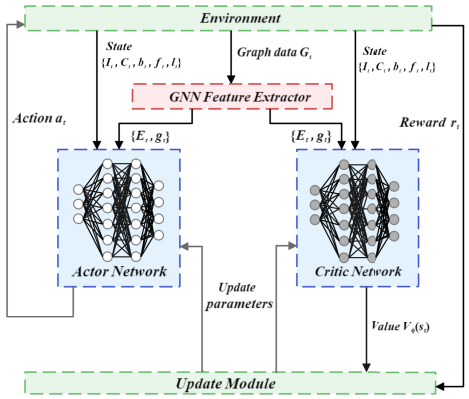


Fig. 3. GNN-based PPO structural framework diagram.

To enable cooperative caching that optimizes the overall system performance, we employ an MARL approach, specifically MAPPO to solve **P2**. Its clipping mechanism effectively limits policy update magnitudes, preventing drastic fluctuations and ensuring efficient, stable learning and convergence. As an actor-critic variant, MAPPO leverages a centralized training with decentralized execution (CTDE) framework. This framework enables UAVs to train with global information while executing decisions independently, making MAPPO highly suitable for systems. Section B detail the definitions of the MDP's state, action, and reward, as well as the design of the MAPPO algorithm.

B. MAPPO-Based Content Caching Strategy

1) State Space: At decision time t, the system's global state S_t aims to provide each UAV $u_i \in U$, acting as an agent, with comprehensive information needed for decision-making. The global state S_t is composed of all UAVs' local states S_t^U and other global information as follows:

$$S_t = \left\{S_t^U, \mathbf{p_t}, \mathbf{q_t}\right\} \tag{25}$$

where $S_t^U = \{s_t^{u_1}, \ldots, s_t^{u_i}, \ldots, s_t^{|U|}\}$, and each UAV u_i's local state s_t^i is defined as: $s_t^{u_i} = \{r_t^{u_i}, c_t^{u_i}, n_t^{u_i}\}$. $r_t^{u_i}$ represents local request features, which describe user content requests within the UAV's service area. $c_t^{u_i}$ describes UAV u_i's current content caching state at time t. The initial distribution of caching state at the beginning of each decision cycle is determined by a pre-caching process using an LSTM prediction model. Specifically, we leverage the efficient aggregation structure built in Section IV to train an LSTM model to predict future content popularity. This prediction directly guides pre-caching decisions, offering a high-quality starting point for subsequent MARL agents to perform real-time and fine-grained cache replacement. $n_t^{u_i}$ represents neighbor caching features, aggregating the caching states from the neighboring nodes to provide cooperative information. $\mathbf{p}_t$ is the global content popularity, $\mathbf{q}_t$ is the global content request.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:23:03 UTC from IEEE Xplore. Restrictions apply.

2) Action Space: At each decision step t, UAV u_i must independently perform a local action $a_t^{u_i}$ to select appropriate content for cache update. The local actions of all UAVs collectively form the joint action as

$$a_t = \left\{a_t^{u_1}, \ldots, a_t^{u_i}, \ldots, a_t^{u_M}\right\}. \tag{26}$$

3) Reward: After executing the joint action a_t, the environment returns a global reward signal r_t, which is shared among all agents to incentivize effective collaboration. The reward function can be expressed as

$$r_t = w_h \cdot H_t - w_{\mathrm{rl}} \cdot L_t \tag{27}$$

where H_t and L_t represent the network-wide average cache hit rate and average request latency at the current decision step t, respectively. This design guides all UAVs to learn toward improving the overall QoE of the entire system.

4) Algorithm Design: To solve the aforementioned MDP in a multiagent environment, we designed a collaborative algorithm based on PPO. Because each UAV must make independent decisions while effective cooperation requires a global perspective, we adopt the CTDE framework. Its core idea is parameter sharing: all UAV agents share the same set of parameters for both the actor and critic networks. This allows us to maintain and update a single global model during training. Under CTDE with parameter sharing, we maintain and update a single global model during training; during execution, agents act independently using the shared parameters. As all agents learn from this shared network, their policies tend to converge toward coordinated behaviors, enabling effective collaboration.

During the centralized training phase, the shared policy network receives the complete global state S_t as input. Its output is a local action probability distribution $\pi_{\theta_{\mathrm{MA}}}(a_t^{u_i} \mid S_t)$ for each UAV u_i. Similarly, the shared value network also receives the global state S_t and outputs a single global state value estimate $V_{\phi_{\mathrm{MA}}}(S_t)$. In the execution phase, each UAV u_i independently samples its caching action based on the local probability distribution generated for it. The training process occurs on a centralized node capable of collecting experience data from all agents. The core of the training remains PPO's update mechanism but with all calculations based on global information. The advantage function A_t is computed using the global reward r_t and the shared value estimate $V_{\phi_{\mathrm{MA}}}(S_t)$. The parameters of the shared policy network $\pi_{\theta_{\mathrm{MA}}}$ and value network $V_{\phi_{\mathrm{MA}}}$ are then updated using the joint action probabilities and global returns of all agents.

Under parameter sharing, the joint policy gradient is

$$\nabla_{\theta_{\mathrm{MA}}} \chi(\theta_{\mathrm{MA}}) = \mathbb{E}_{a_t \sim \pi_{\theta_{\mathrm{MA}}}(\cdot|S_t)} \left[\sum_{i=1}^{M} \nabla_{\theta_{\mathrm{MA}}} \log \pi_{\theta_{\mathrm{MA}}}\left(a^i \middle| S_t\right) A_t \right] \tag{28}$$

where $\pi_{\theta_{\mathrm{MA}}}(\cdot \mid S_t)$ is the shared policy of all UAV agents. a_t represents the joint action taken by all agents. A_t is the advantage function at time step t, which is computed using the global reward r_t and global state value $V_{\phi_{\mathrm{MA}}}(S_t)$. The gradient of the policy is computed with respect to the parameters θ_{MA} shared by all agents, guiding them to take actions that maximize the expected cumulative reward.

In summary, using CTDE with a parameter-sharing actor–critic architecture produces a collaborative multiagent system. Each UAV makes local decisions distributively while learning to cooperate via the centrally trained shared policy, ultimately optimizing caching performance and enhancing user QoE. The detailed steps of the proposed DFL-based MAPPO collaborative caching algorithm (DFL-MAPPO) appear in Algorithm 2.

Algorithm 2 DFL-Based MAPPO Collaborative Caching Algorithm (DFL-MAPPO)

Input: $\mathcal{F}$, U, V and U_P^*.
Initialization: γ_{MA}, λ_{MA}, ϵ_{MA}, $N_{\max}$, $T_{\max}$, $M_{\max}$, Θ, κ, $\pi_{\theta_{\mathrm{MA}}}$, $\pi_{\phi_{\mathrm{MA}}}$.
$\{c^{u_1}, \ldots, c^{u_i}\}_{u_i \in U} \leftarrow$ *DFL Training*(U, V, U_P^*);
for $n = 1$ to $N_{\max}$ **do**
  for $t = 1$ to $T_{\max}$ **do**
    for each agent $u_i \in U$ **do**
      Construct state $\{r_t^{u_i}, c_t^{u_i}, n_t^{u_i}\}$;
      Select action $a_t^{u_i}$ based on actor network;
      Estimate $V_{\phi_{\mathrm{MA}}}(S_t)$;
    end for
    Execute joint action $a_t = \{a_t^1, \ldots, a_t^M\}$, receive reward r_t and state S_{t+1};
    Store experience $(S_t, a_t, r_t, V_{\phi_{\mathrm{MA}}}(S_t), \log \pi_{\theta_{\mathrm{MA}}}(a_t|S_t))$ for each agent;
  end for
  for each UAV $u_i \in U$ agent **do**
    Calculate GAE, returns, probability ratio, and clipped surrogates same as Algorithm 1, with parameters γ_{MA}, λ_{MA}, $\pi_{\theta_{\mathrm{MA}}}$ and $\pi_{\phi_{\mathrm{MA}}}$;
    Calculate the joint policy gradient via Eq. (28);
    Update actor-critic networks with parameter ϵ_{MA}, $\pi_{\theta_{\mathrm{MA}}}$ and $\pi_{\phi_{\mathrm{MA}}}$;
  end for
end for
Output: Optimized caching strategy $\mathcal{C}^*$.
Function *DFL Training*(U, V, U_P^*)
for $m = 1$ to $M_{\max}$ **do**
  for $u_i \in U$ **do**
    for $v_j \in V$ **do**
      Train local model κ_{v_j} using local data;
      Send model updates to the corresponding UAV;
    end for
    Aggregate model updates from all vehicles that are in the communication area;
    Perform DFL aggregation based on the optimized routing path U_P^*;
    Update and distribute the model to each vehicle;
    Update u_i's cache state c^i based on the trained model;
  end for
end for
Output: Cache state c^{u_i} for $u_i \in U$.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:23:03 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PARAMETERS SETUP

Parameter	Value
N	[15,50]
M	[5,40]
a_1	9.61
a_2	0.28
η_{LoS}	1 dB
η_{NLoS}	20 dB
p_{u_i}	0.4 W
B	40 MHz
ϑ	2 GHz
Θ	[5,40] GB

VI. SIMULATION RESULTS

A. Simulation Settings

To evaluate the effectiveness of the proposed GPPO-R and DFL-MAPPO algorithms, we set a severe traffic congestion on a road, with vehicles moving at an average speed of 2 m/s. The parameters involved in the model are detailed in Table I.

For the simulation dataset, we use the MovieLens 1M dataset, which is a standard dataset widely used in recommendation research [47]. It contains 1×10^6 entries comprising the rating records of 6040 users for 3883 movies, spanning the period from November 1997 to April 2000. It is divided into three parts: user information, movie information, and rating records. To simulate content requests, we assume that the rated movies represent the content requested by users.

We compare the performance of five algorithms for routing: the proposed *GPPO-R*, GNN-enhanced deep Q-network (*GDQN-R*), PPO (*PPO-R*), GNN (*GNN-R*), and MLP-enhanced PPO (*MPPO-R*). Each algorithm uses a different approach to model and optimize the communication paths within the network.

1) *GDQN-R:* It combines a GNN encoder with deep Q-network (DQN) to learn optimal paths from topology embeddings.
2) *PPO-R:* It applies PPO without GNN enhancements, relying solely on reward-driven policy updates.
3) *GNN-R:* It uses a GNN to encode network state into low-dimensional vectors and directly generates routing sequences via forward propagation, without RL.
4) *MPPO-R:* It uses an MLP encoder as the front-end feature extractor within PPO.

B. Discussion of GPPO-R

1) Convergence of GPPO-R: Fig. 4(a) shows the training rewards, which climb sharply after several episodes and then plateau, demonstrating rapid learning and policy convergence. Fig. 4(b) illustrates the synchronization latency, which spikes early before settling into a lower steady state as aggregation becomes more efficient. Fig. 4(c) shows communication energy consumption, starting high during exploration and gradually declining to a stable baseline. Fig. 4(d) depicts load energy consumption, which likewise fluctuates initially and then stabilizes, reflecting optimized load balancing and task scheduling.

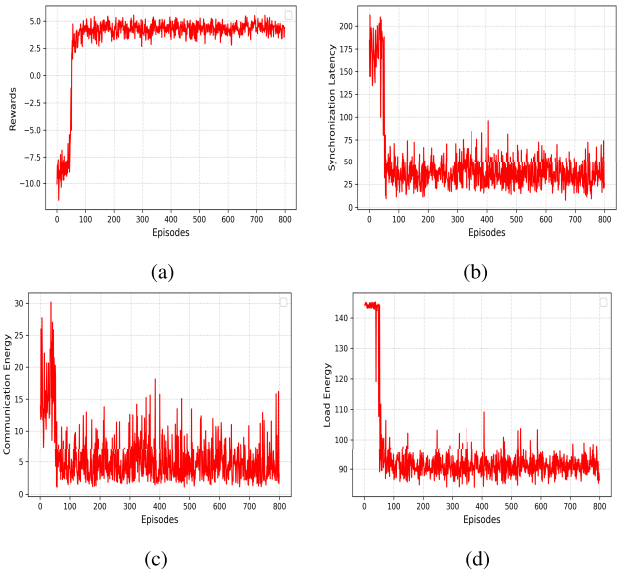


Fig. 4. Training metrics. (a) Rewards. (b) Synchronization latency. (c) Communication energy. (d) Load energy.

2) Advantages of the GNN Encoder in PPO: Fig. 5 shows the smoothed training trajectories for the proposed GPPO-R, MPPO-R, and PPO-R baseline under three network sizes, $M \in \{15, 25, 35\}$. GPPO-R converges faster, attains higher steady-state reward, and exhibits lower training variance than the baselines, and these advantages increase with M. The main reason is that larger graphs produce richer interactions and thus more informative samples. GNN message passing accelerates credit assignment along edges, and parameter sharing enlarges the effective sample pool for the shared encoder. In MPPO-R, the unstructured MLP front end cannot exploit locality or relational structure and its input dimensionality grows with M, which tends to dilute informative signals and slow learning. These trends are particularly pronounced for larger networks. The PPO-R baseline struggles to converge within the training budget for $M = 25$ and $M = 35$, exhibiting slower upward trends. MPPO-R attains intermediate behavior but also requires more episodes than GPPO-R. Together, these observations confirm that it provides a useful relational inductive bias to improve convergence speed and enhance sample efficiency.

3) Validation of GPPO-R: Fig. 6 illustrates how synchronization latency, communication energy, and load energy scale with the number of UAVs under the proposed GPPO-R. The stacked bars show the cumulative value of each metric, with the total plotted as a line. As the number of UAVs rises, the total cost grows steadily, as synchronization latency (red) dominates the increase, while communication energy (blue) and load energy (green) climb more moderately. These trends imply that adding UAVs expands capacity but incurs disproportionately higher synchronization and communication overhead.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:23:03 UTC from IEEE Xplore. Restrictions apply.

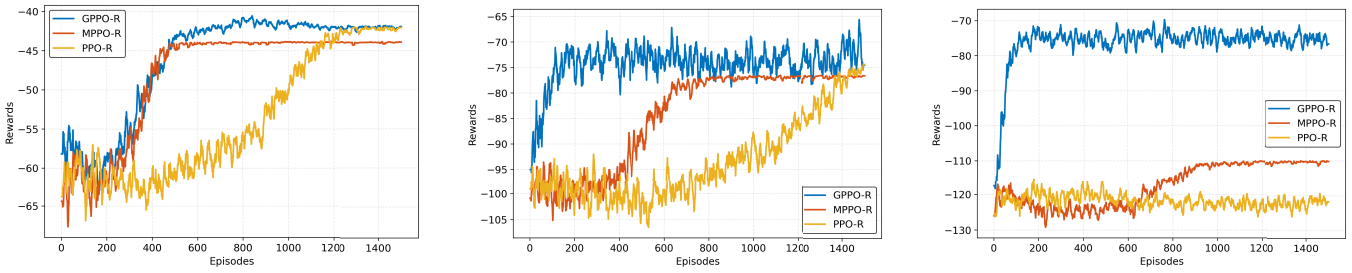


Fig. 5. Training reward curves for GPPO-R, MPPO-R, and PPO-R under three network sizes. (a) $M = 15$. (b) $M = 25$. (c) $M = 35$.

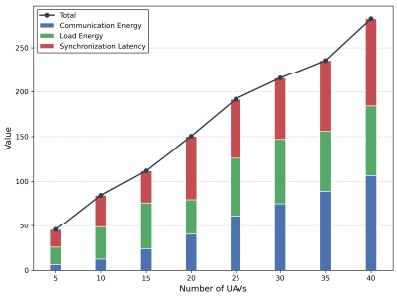


Fig. 6. Overhead scaling with a number of UAVs under GPPO-R.

Fig. 7 illustrates how synchronization latency, communication energy, and load energy scale with the number of UAVs for aggregation structures generated by different algorithms. Fig. 7(a) shows the synchronization latency. GDQN-R achieves the lowest latency at every UAV count, followed by GPPO-R. PPO-R incurs higher delays, and GNN-R shows the worst performance. Fig. 7(b) shows the total energy consumption as the number of UAVs increases. GDQN-R incurs the highest energy consumption across all scales, followed by PPO-R. GPPO-R and GNN-R both consume significantly less energy and remain nearly identical, demonstrating their efficiency in aggregation and synchronization. Fig. 7(c) reports the composite overhead (weighted sum of synchronization latency and total energy). Although GDQN-R delivers the lowest synchronization latency in Fig. 7(a), it incurs the highest energy consumption in Fig. 7(b), resulting in poor balance between latency and energy. PPO-R follows a similar pattern with high overall cost. In contrast, GPPO-R, while not the absolute minimum on either individual metric, achieves the lowest composite overhead. This demonstrates that GPPO-R strikes the best tradeoff between synchronization speed and energy efficiency.

Fig. 8 compares request latency under the DFL-MAPPO caching strategy using different DFL aggregation algorithms. Fig. 8(a) plots average request latency versus the number of UAVs. All four strategies benefit from more UAVs, showing steadily falling latency. GPPO-R consistently achieves the lowest latency for all UAV counts. GDQN-R and GNN-R follow closely, while PPO-R remains the highest. This confirms that the GNN-enhanced PPO aggregation structure best supports low-latency caching. Fig. 8(b) shows latency as cache capacity increases. Every curve trends downward as more cache is available. GPPO-R again leads with the smallest latency across all capacities. GDQN-R and GNN-R improve moderately, and PPO-R improves the least. This highlights GPPO-R's robustness in leveraging cache resources. Fig. 8(c) depicts latency against the number of users. As user load grows, latency for all strategies fluctuates slightly but remains in a narrow band. GPPO-R maintains the lowest values throughout, demonstrating its resilience under heavy request loads. These results show that although other methods can approach GPPO-R under some conditions, GPPO-R consistently provides the best overall latency performance.

To validate robustness under cache-capacity heterogeneity, we conducted two complementary experiments reported in Fig. 9. Fig. 9(a) shows the UAV-heterogeneous condition, where per-UAV caching capacities were independently sampled from $[5, 15]$ GB and uniformly scaled by a factor s. Fig. 9(b) shows the vehicle-heterogeneous condition, where per-vehicle caching capacities were independently sampled from $[3, 8]$ GB and uniformly scaled by the same factor s, while UAV capacities were held fixed. $s \in \{1.00, 1.25, \ldots, 3.00\}$. The heterogeneous results illustrate that increasing the effective cache capacity reduces average request-processing latency. The curves exhibit some point-to-point variability due to random initial draws and finite averaging. Heterogeneity increases latency variance because small caching capacity nodes can intermittently become bottlenecks. Importantly, the proposed GPPO-R consistently attains the lowest latency across heterogeneous settings, demonstrating that the benefits of capacity scaling and the GNN front-end persist in realistic deployments.

C. Discussion of the Proposed DFL-MAPPO

1) Convergence of DFL-MAPPO: Fig. 10 shows the training rewards of the DFL-MAPPO caching strategy. The curve begins deep in negative territory, reflecting an initially undeveloped policy. Over the first 300 episodes, the reward climbs steadily into positive values. Thereafter, it oscillates tightly around zero with modest positive peaks, indicating that learning has plateaued. This trend demonstrates that DFL-MAPPO effectively discovers a near-optimal caching policy and converges after adequate training.

2) Validation of DFL-MAPPO: We compare the performance of four algorithms for caching strategy. Our proposed

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:23:03 UTC from IEEE Xplore. Restrictions apply.

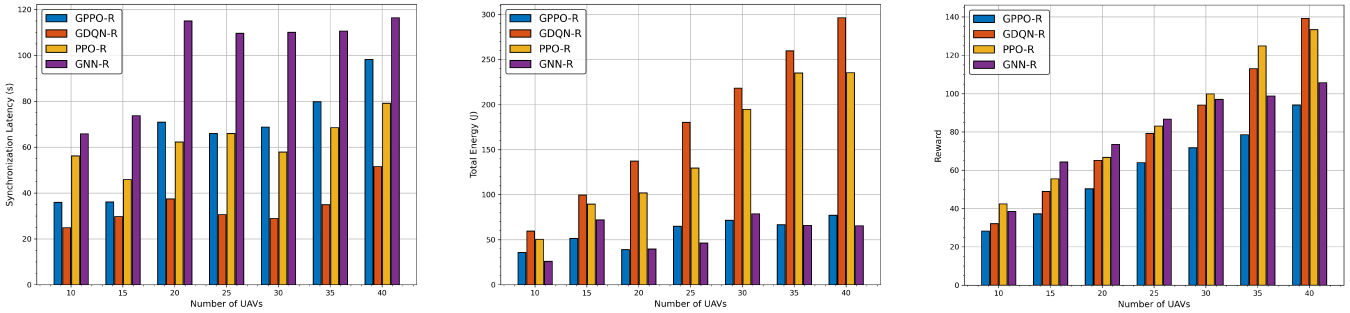


Fig. 7. Comparison of latency and energy consumption by aggregation strategy under varying UAV numbers. (a) Synchronization latency. (b) Total energy. (c) Reward.

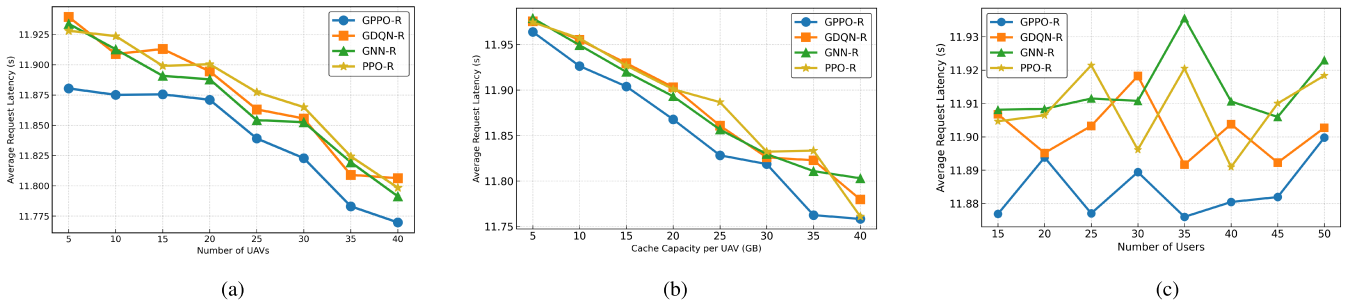


Fig. 8. Comparison of average request latency by aggregation strategy under different settings. (a) Number of UAVs. (b) Cache capacity per UAV (GB). (c) Number of users.

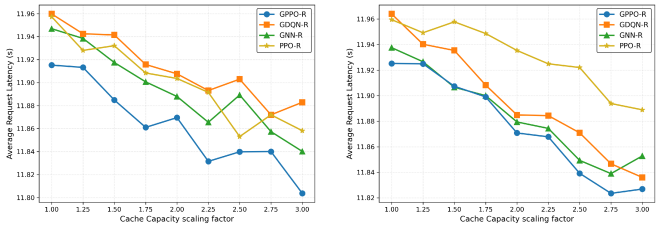


Fig. 9. Cache-capacity scaling under two-sided heterogeneity. (a) UAV side. (b) Vehicle side.

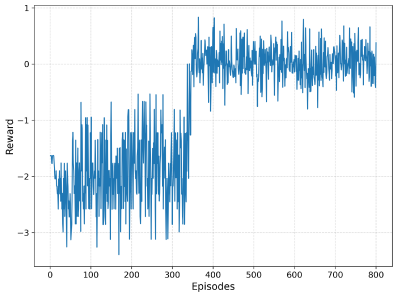


Fig. 10. Training reward of DFL-MAPPO.

neighbor-aware *DFL-MAPPO*, DFL-MAPPO without considering neighbor node cache information (*DFL-MAIPPO*), LRU (*DFL-LRU*), and LFU (*DFL-LFU*).

1) *DFL-MAIPPO:* This algorithm leverages the DFL approach, combined with MAPPO. However, this variant does not consider the caching state of neighboring UAVs, meaning that each UAV acts independently without leveraging the cache state of its neighbors.
2) *DFL-LRU:* This traditional caching strategy prioritizes keeping the most recently requested content in the cache.
3) *DFL-LFU:* Similar to LRU, LFU caches content based on how frequently it is requested.

Fig. 11 compares request latency under GPPO-R using different caching algorithms. Fig. 11(a) displays latency versus the number of UAVs. More UAVs reduce latency for all methods, but DFL-MAPPO makes the most effective use of additional UAVs. DFL-MAIPPO also improves performance but at a slower rate, whereas DFL-LRU and DFL-LFU lag behind due to their noncooperative policies. Fig. 11(b) shows the latency versus cache capacity. Increasing cache capacity reduces latency for every strategy. DFL-MAPPO exhibits the steepest decline and lowest latency across all capacities. DFL-MAIPPO obtains the second-largest improvement, while both DFL-LRU and DFL-LFU improve only modestly and converge at high capacities, highlighting the value of adaptive learning over static replacement. Fig. 11(c) plots average request latency versus the number of users. As the number of users increases, all methods exhibit slight latency growth. DFL-MAPPO consistently achieves the lowest latency, demonstrating strong resilience under heavy demand. DFL-MAIPPO follows, showing that independent learning still outperforms traditional schemes. DFL-LFU and DFL-LRU perform worst, with DFL-LFU marginally better than DFL-LRU at higher user loads.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:23:03 UTC from IEEE Xplore. Restrictions apply.

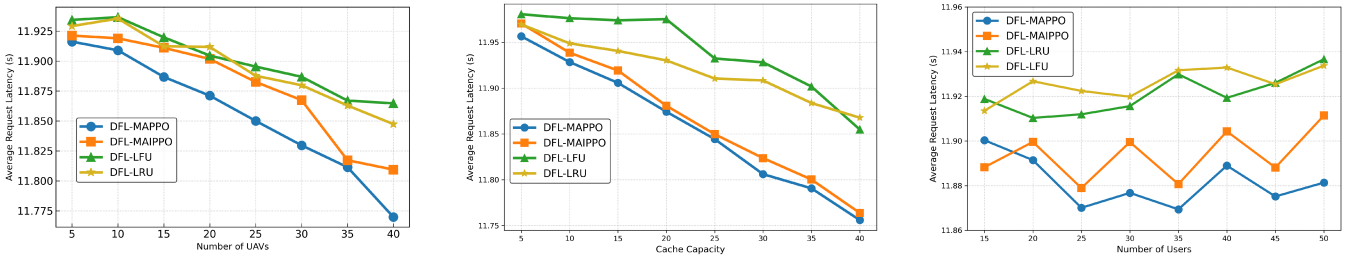


Fig. 11. Comparison of average request latency by caching strategy under different settings. (a) Number of UAVs. (b) Cache capacity (GB). (c) Number of users.

VII. Conclusion

In this article, we present a unified framework for UAV-assisted IoV that jointly optimizes DFL aggregation and the cooperative edge-caching strategy. First, the proposed GPPO-R algorithm embeds dynamic network topology by introducing a GNN into each PPO agent, enabling it to converge rapidly to near-optimal aggregation structures. GPPO-R consistently achieves the lowest composite overhead, balancing model synchronization latency and system energy consumption. Second, the proposed DFL-MAPPO caching strategy incorporates neighbor-state sharing, which yields stable convergence and outperforms its independent counterpart DFL-MAIPPO as well as traditional LRU and LFU schemes. Across varying numbers of UAVs, cache capacities, and user loads, DFL-MAPPO reduces the request latency and overall system overhead compared to the baselines, demonstrating its effectiveness and scalability in dynamic UAV networks. In the future, we plan to extend the proposed GPPO-R and DFL-MAPPO frameworks to incorporate explicit mobility models and mobility-resilient protocol components, to evaluate their impact across a wider range of speed regimes and handover dynamics.

References

[1] K. Raja et al., "An efficient 6G federated learning-enabled energy-efficient scheme for UAV deployment," *IEEE Trans. Veh. Technol.*, vol. 74, no. 2, pp. 2057–2066, Feb. 2025.

[2] W. Wang, Y. Liu, Y. Dai, and Y. He, "Joint cooperative caching and power control for UAV-assisted Internet of Vehicles," *Sci. Rep.*, vol. 14, no. 1, p. 9341, Apr. 2024.

[3] X. Xia, S. M. M. Fattah, and M. A. Babar, "A survey on UAV-enabled edge computing: Resource management perspective," *ACM Comput. Surv.*, vol. 56, no. 3, pp. 1–36, Mar. 2024.

[4] C. Yang et al., "RaftFed: A lightweight federated learning framework for vehicular crowd intelligence," 2023, *arXiv:2310.07268*.

[5] J. Liao, B. Jiang, P. Zhao, L. Ning, and L. Chen, "Unmanned aerial vehicle-assisted federated learning method based on a trusted execution environment," *Electronics*, vol. 12, no. 18, p. 3938, Sep. 2023.

[6] X. He et al., "QoE-driven proactive caching with DRL in sustainable cloud-to-edge continuum," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 1–13, Oct. 2025.

[7] D. Lee et al., "On the existence of a spectrum of policies that subsumes the least recently used (LRU) and least frequently used (LFU) policies," in *Proc. ACM SIGMETRICS Int. Conf. Meas. Model. Comput. Syst.*, 1999, pp. 134–143.

[8] X. Li, J. Liu, X. Chen, J. Wang, and M. Pan, "Caching on the sky: A multiagent federated reinforcement learning approach for UAV-assisted edge caching," *IEEE Internet Things J.*, vol. 11, no. 17, pp. 28213–28226, Sep. 2024.

[9] H. Wu, B. Wang, H. Ma, X. Zhang, and L. Xing, "Multiagent federated deep-reinforcement-learning-based collaborative caching strategy for vehicular edge networks," *IEEE Internet Things J.*, vol. 11, no. 14, pp. 25198–25212, Jul. 2024.

[10] G. T. Maale, N. A. E. Kuadey, Y. Arafat, K. Thomas, G. Sun, and G. Liu, "Multi-task learning for UAV trajectory and caching with federated cloud-assisted knowledge distillation," *IEEE Trans. Netw. Service Manage.*, vol. 22, no. 3, pp. 2516–2533, Jun. 2025.

[11] C. Shang, D. T. Hoang, M. Hao, D. Niyato, and J. Yu, "Energy-efficient decentralized federated learning for UAV swarm with spiking neural networks and leader election mechanism," *IEEE Wireless Commun. Lett.*, vol. 13, no. 10, pp. 2742–2746, Oct. 2024.

[12] S. Huang, H. Yao, X. Wang, T. Mai, Z. Wang, and S. Guo, "Graph attentional based agglomerative cluster for UAV swarm networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 4, pp. 1–18, Jul. 2025.

[13] M. Gillani, H. A. Niaz, A. Ullah, M. U. Farooq, and S. Rehman, "Traffic aware data gathering protocol for VANETs," *IEEE Access*, vol. 10, pp. 23438–23449, 2022.

[14] R. Lai et al., "Energy-efficient scheduling in UAV-assisted hierarchical wireless sensor networks," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 20194–20206, Jun. 2024.

[15] T. Devred, M. Wahl, and P. Sondi, "A comparison of backbone and mesh clustering strategies for collaborative management of 6G vehicle-to-vehicle exchanges," *Electronics*, vol. 13, no. 3, p. 572, Jan. 2024.

[16] X. Zhu, S. Yu, J. Wang, and Q. Yang, "Efficient model compression for hierarchical federated learning," 2024, *arXiv:2405.17522*.

[17] L. Chen, W. Liu, Y. Chen, and W. Wang, "Communication-efficient design for quantized decentralized federated learning," *IEEE Trans. Signal Process.*, vol. 72, pp. 1175–1188, 2024.

[18] L. Liu, J. Zhang, S. Song, and K. B. Letaief, "Hierarchical federated learning with quantization: Convergence analysis and system design," *IEEE Trans. Wireless Commun.*, vol. 22, no. 1, pp. 2–18, Jan. 2023.

[19] V. M. R. Tummala, A. Hazra, A. Kalita, and M. Gurusamy, "Cluster based pseudo hierarchical decentralized federated learning in UAV networks," in *Proc. IEEE 100th Veh. Technol. Conf. (VTC-Fall)*, Washington, DC, USA, Oct. 2024, pp. 1–5.

[20] D. Pan, M. A. Khoshkholghi, and T. Mahmoodi, "Decentralized federated learning methods for reducing communication cost and energy consumption in UAV networks," in *Mobile Computing, Applications, and Services*, J. Taheri, M. Villari, and A. Galletta, Eds., Cham, Switzerland: Springer, 2023, pp. 18–30.

[21] Z. Tong, J. Wang, X. Hou, J. Chen, Z. Jiao, and J. Liu, "Blockchain-based trustworthy and efficient hierarchical federated learning for UAV-enabled IoT networks," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34270–34282, Nov. 2024.

[22] M. F. Reza, R. Jahani, R. Jin, and H. Dai, "Distribution-aware mobility-assisted decentralized federated learning," 2025, *arXiv:2505.18866*.

[23] D. Chen, T. Deng, J. Jia, S. Feng, and D. Yuan, "Mobility-aware decentralized federated learning with joint optimization of local iteration and leader selection for vehicular networks," *Comput. Netw.*, vol. 263, May 2025, Art. no. 111232.

[24] Z. Tong, J. Wang, J. Chen, X. Zhang, H. Du, and J. Liu, "AirFRL: Topology-aware decentralized federated reinforcement learning for UAV networks," in *Proc. IEEE 101st Veh. Technol. Conf. (VTC-Spring)*, Jun. 2025, pp. 1–6.

[25] Y. Wang, Z. Wen, Y. Li, and B. Cao, "Learn to collaborate in MEC: An adaptive decentralized federated learning framework," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 14071–14084, Dec. 2024.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:23:03 UTC from IEEE Xplore. Restrictions apply.

[26] G. Tan, H. Yuan, H. Hu, S. Zhou, and Z. Zhang, "A framework of decentralized federated learning with soft clustering and 1-bit compressed sensing for vehicular networks," *IEEE Internet Things J.*, vol. 11, no. 13, pp. 23617–23629, Jul. 2024.
[27] F. Solat, J. Lee, and D. Niyato, "Split federated learning-empowered energy-efficient mobile traffic prediction over UAVs," *IEEE Wireless Commun. Lett.*, vol. 13, no. 11, pp. 3064–3068, Nov. 2024.
[28] G. Hu, D. Zhu, J. Shen, J. Hu, J. Han, and T. Li, "FedBeam: Reliable incentive mechanisms for federated learning in UAV-enabled Internet of Vehicles," *Drones*, vol. 8, no. 10, p. 567, Oct. 2024.
[29] G. Yu, Y. He, J. Wu, Z. Chen, and J. Pan, "Mobility-aware proactive edge caching for large files in the Internet of Vehicles," *IEEE Internet Things J.*, vol. 10, no. 13, pp. 11293–11305, Jul. 2023.
[30] W. Yang, X. Liu, L. He, Y. Du, and Y. Chen, "A distributed clustering algorithm based on coalition game for intelligent vehicles," 2025, *arXiv:2503.08416*.
[31] H. Yan, H. Li, X. Xu, and M. Bilal, "UAV-enhanced service caching for IoT systems in extreme environments," *IEEE Internet Things J.*, vol. 11, no. 16, pp. 26741–26750, Aug. 2024.
[32] R. Zhou et al., "User preference oriented service caching and task offloading for UAV-assisted MEC networks," *IEEE Trans. Services Comput.*, vol. 18, no. 2, pp. 1097–1109, Mar. 2025.
[33] W. Wang, X. Xu, M. Bilal, M. Khan, and Y. Xing, "UAV-assisted content caching for human-centric consumer applications in IoV," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 927–938, Feb. 2024.
[34] E. M. Diallo, R. Chai, A. B. M. Adam, G. A. Bayessa, C. Liang, and Q. Chen, "Long-term energy consumption minimization based on UAV joint content fetching and trajectory design," *Sensors*, vol. 25, no. 3, p. 898, Feb. 2025.
[35] K. Long, C. Li, K. Jiang, and S. Wan, "Improved AFSA-based energy-aware content caching strategy for UAV-assisted VEC," *IEEE Trans. Sustain. Comput.*, vol. 10, no. 2, pp. 366–377, Mar. 2025.
[36] P. Ou, N. Chen, and L. Yang, "PDRL-CM: An efficient cooperative caching management method for vehicular networks based on deep reinforcement learning," *Ad Hoc Netw.*, vol. 176, Sep. 2025, Art. no. 103888.
[37] Y. Liu, C. Yang, X. Chen, and F. Wu, "Joint hybrid caching and replacement scheme for UAV-assisted vehicular edge computing networks," *IEEE Trans. Intell. Vehicles*, vol. 9, no. 1, pp. 866–878, Jan. 2024.
[38] P. Zhang et al., "UAV communication in space–air–ground integrated networks (SAGINs): Technologies, applications, and challenges," *Drones*, vol. 9, no. 2, p. 108, Feb. 2025.
[39] Z. Li, Y. Liu, K. G. Shin, J. Liu, and Z. Yan, "Interference steering to manage interference in IoT," *IEEE Internet Things J.*, vol. 6, no. 6, pp. 10458–10471, Dec. 2019.
[40] M. Mozaffari, W. Saad, M. Bennis, Y.-H. Nam, and M. Debbah, "A tutorial on UAVs for wireless networks: Applications, challenges, and open problems," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2334–2360, 3rd Quart., 2019.
[41] Y. Zeng, R. Zhang, and T. J. Lim, "Wireless communications with unmanned aerial vehicles: Opportunities and challenges," *IEEE Commun. Mag.*, vol. 54, no. 5, pp. 36–42, May 2016.
[42] C. Li, Y. Zhang, and Y. Luo, "A federated learning-based edge caching approach for mobile edge computing-enabled intelligent connected vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 3, pp. 3360–3369, Mar. 2023.
[43] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proc. 34th Int. Conf. Mach. Learning*, vol. 70, 2017, pp. 1263–1272.
[44] P. W. Battaglia et al., "Relational inductive biases, deep learning, and graph networks," 2018, *arXiv:1806.01261*.
[45] S. Sukhbaatar, A. Szlam, and R. Fergus, "Learning multiagent communication with backpropagation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, 2022, pp. 2252–2260.
[46] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
[47] F. M. Harper and J. A. Konstan, "The MovieLens datasets: History and context," *ACM Trans. Interact. Intell. Syst.*, vol. 5, no. 4, pp. 1–19, Jan. 2016.



Suidan Yuan received the B.E. degree from Guangdong University of Technology, Guangzhou, China, in 2023, where she is currently pursuing the M.S. degree with the School of Automation.

Her research interests include federated learning, wireless communication networks, and intelligent edge computing.



Chao Yang received the Ph.D. degree in signal and information processing from South China University of Technology, Guangzhou, China, in 2013.

From July 2014 to July 2016, he was a Research Associate at the Department of Computing, The Hong Kong Polytechnic University, Hong Kong. He is currently with the School of Automation, Guangdong University of Technology, Guangzhou. His research interest focuses on VANETs, edge computing, and smart grids.



Jiajie Zhou received the B.E. degree from Guangdong University of Technology, Guangzhou, China, in 2024, where he is currently pursuing the M.S. degree with the School of Automation.

His research interests include wireless communication networks and intelligent edge computing.



Shengli Xie (Fellow, IEEE) received the M.S. degree in mathematics from Central China Normal University, Wuhan, China, in 1992, and the Ph.D. degree in control theory and applications from the South China University of Technology, Guangzhou, China, in 1997.

He is currently a Full Professor and the Head of the Institute of Intelligent Information Processing, Guangdong University of Technology, Guangzhou, and the Director of the Laboratory for Intelligent Information Processing and the Director of Guangdong–Hong Kong–Macao Joint Laboratory for Smart Discrete Manufacturing. He has authored or co-authored two books and over 150 scientific articles in journals and conference proceedings. His research interests include wireless networks, automatic control, and blind signal processing.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on May 20,2026 at 02:23:03 UTC from IEEE Xplore. Restrictions apply.